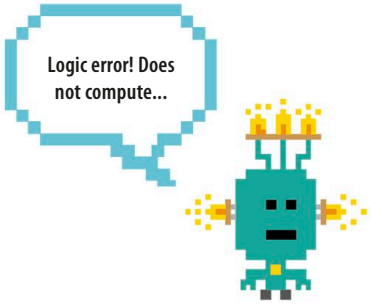
Logic errors

Sometimes you can tell something has gone wrong even if Python hasn't given you an error message, because your program isn't doing what you expected. It could be that you've got a logic error. You may have typed in the code correctly, but if you missed an important line or put the instructions in the wrong order it won't run properly.

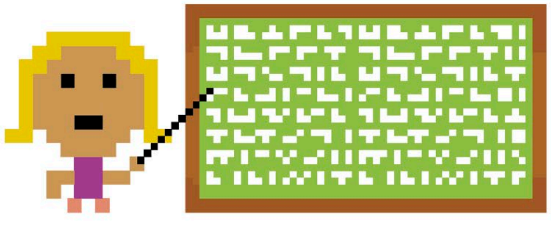



```
print('Oh no! You have lost a life')
print(lives)
lives = lives - 1
```

All the lines of code are correct, but two are in the wrong order.

◁ **Can you spot the bug?**

This code will run with no error messages, but there's a logic error in it. The value of `lives` is shown on the screen before the number of lives is reduced by one. The player of this game will see the wrong number of lives remaining! To fix it, move the instruction `print(lives)` to the end.



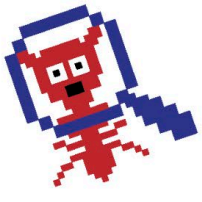
◁ **Line by line**

Logic errors can be tricky to find, but as you get more experienced you'll get good at tracking them down. Try to identify logic errors by checking your code slowly, line by line. Be patient and take your time – you'll find the problem in the end.

EXPERT TIPS

Bug-busting checklist

Sometimes you might think that you'll never get a program to work, but don't give up! If you follow the tips in this handy checklist, you'll be able to identify most errors.



Ask yourself...

- If you build one of the projects in this book and it doesn't work, check that the code you've typed matches the book exactly.
- Is everything spelled correctly?
- Do you have unnecessary spaces at the start of a line?
- Have you confused any numbers for letters, such as 0 and O?
- Have you used upper-case and lower-case letters in the right places?
- Do all open brackets have a matching closing bracket? ( ) [ ] { }
- Do all single and double quotes have a matching closing quote? ' ' " "
- Have you asked someone else to check your code against the book?
- Have you saved your code since you last made changes?